

SER 540/494 - Project 1

[10 Points]

Description

In this assignment you will create a Java program that is able to simulate/replay sensor values of a test drive with the correct timing. The sensor values and their timing are taken from a CAN trace file (parsed, decoded and stored in a *CANTrace* object in a previous assignment). In addition, the GPS positions are taken from a separate file, which contains the recorded GPS coordinates of the test vehicle during the test drive.

A GUI will show all most recent sensor values and current GPS coordinates for the entire duration of the simulated test drive. Additionally, the current test drive simulation time is shown. This simulation must match the timing exactly as recorded (1 second during the test drive must be 1 second in the simulation).

For this assignment, you can (and should) use the code from the previous CAN trace assignment and extend it.

Tasks

This first project is divided into three parts

1. Parsing GPS
2. Simulator
3. Simulator GUI

Part 1: Parsing GPS

Download the GPS trace file from Canvas. Create a class named **GPSParser** that accepts this file as argument, extracts the GPS coordinates (no decoding required, since they are in plain text) and returns a *GPSTrace* object (see below).

The GPS coordinates are reported every second (1000 ms) and the first GPS coordinate was transmitted at the start of the test drive (offset 0 ms). Use this to generate a time offset for each coordinate recorded during the test drive.

Create a class called *GPScoordinates*, which includes the latitude, longitude and the time offset. Then, use any Java collection you deem suitable to create a GPS specific data structure called *GPSTrace*, which contains all *GPScoordinates* elements of the test drive. Make sure there is an easy way to iterate through this collection and retrieve all coordinates with an ascending time offset.

Part 2: Simulator

Extend the existing *CANSimulation* class from the previous CAN trace assignment and implement a public method with the following signature **void startSimulation()**. This method uses the local *CANTrace* and *GPSTrace* object and will be responsible to send these sensor values with the correct timing to a different class. Make sure this method only executes, if *CANTrace* and *GPSTrace* objects with data exist (parsing of CAN and GPS was done successfully).

The simulator must be able to send the sensor values to a different class (calling a function; more details in the next paragraph) with the correct timing (e.g. if the sensor value X was recorded with time offset 1760 ms in the CAN trace, then this sensor value must be "delivered" to the *receive* function 1760 ms after the user started the simulation; in other words: the simulation must run in real time). This class must allow the user to start the simulation. If the simulation of the test drive is done (reached the end) and the user starts it again, the simulation starts over from the beginning. Further control of the simulation is not required, e.g., no need to implement a pause/resume or similar.

The sent sensor values are processed by a different class called **SensorDataReceiver**. This class will have only one public method, which can be called to send/receive all sensor data (CAN data and the GPS coordinates) called **void receiveSensorValues(...)** and it has three arguments: sensor value, time offset, identifier (to identify which sensor value was sent over).

Note: Having only one argument for a sensor value means this function needs to be called multiple times in some cases, e.g. in order to receive the full GPS coordinates (call once for latitude, call again for longitude). *Note:* You can create other methods, but only the one function described above is allowed to receive the sensor values. *Note:* The simulator only sends the sensor values to the receiver. If you need the sensor values in another class, the *SensorDataReceiver* class must forward them accordingly.

In the **SensorDataReceiver** class, output the current time of the test drive (starting at 0) and the latest value of each sensor in the console. Do not create a new line for every value. Instead, update the value within a single line (use carriage return). If some sensors are missing values in the beginning, print a dash "-" instead of the value, until the first value was received. *Note:* Remember that the first GPS position is transmitted at time offset 0 ms.

The simulated test drive must to match the test drive duration and the sensor timing of the parsed values.

A possible output format could be:

Current Time	Vehicle Speed	SteerAngle	YawRate	LatAccel	LongAccel	GPS Lat/Long
12,345 ms	123.45 km/h	123.45 deg	...			

Figure 1: Example of console output.

Part 3: Simulator GUI

Create a GUI for the simulator, which allows to start the simulation of the test drive with a button. Once the test drive is started, display the current test drive time and the latest value of each sensor in the GUI (similar to the console output of the previous part).

Once the test drive is completed (last value was displayed), stop the current test drive time and keep displaying the last sensor values until the user clicks the button again and starts the test drive again from the beginning. Make sure to reset the displayed values for the following test drive simulation.

This GUI should be in a separate class called **SimulationGUI** and only interface with the **SensorDataReceiver** class to receive the sensor values. Create the required functions as you need them for your GUI.

Important: You are allowed to create this GUI in any language/way you want and it does not have to be implemented in Java (Only the GUI! Everything else must be implemented in Java.). You can pick different languages or technologies as long as you interface with the Java implementation (the **SensorDataReceiver** class) and use the transmitted sensor data from there. If you want to stick to Java, you are also free to choose any way of creating a GUI (e.g., Swing).

Deliverables

This assignment must be submitted on Canvas before the due date (see Canvas) with the following deliverables.

1. Submit a document *.pdf* file that contains the following:
 - a) A list of all implemented classes with a brief description what the purpose / function of that class is
 - b) A list of all group members and for each member what they worked on (e.g. which functions, classes, and similar the member worked on)
 - c) A screenshot of the console output (should show proper sensor values at any point during the simulation)
 - d) A screenshot of the GUI while simulating (should show proper sensor values at any point during the simulation)
2. Submit the code in a zip file including a working Gradle build script. Include a *readme* that explains how to compile and run your project from console via the Gradle script. Make sure to write sufficient comments, which explain every line of code (can be single comments per line or block comments to explain a section). **Important:** If the project does not compile and run via the Gradle script, you will receive 0 points for the coding part (majority of points).
3. Record a video (e.g. use screen capture software like OBS or use Zoom) where you explain your code briefly, compile the code and show the data output in the console and the GUI for one simulation of the test drive. Maximum video length is 240 seconds - your video must be shorter or equal.

Note: Only one group member has to present in the video, although a shared effort is appreciated (all members presenting a part of the video).

Upload this video to YouTube (**you must upload the video as unlisted**) and copy the link to your video into the comment section of the submission.